

THE JUDGE QUEUE: a playbook for building a personal assistant's UI around one human decider

A working guide for a code assistant. Everything here was learned building a large creative project where one person judged everything on a phone and a fleet of AI chats did the work. Read it once, then hand the checklists to your code bot. No product names, no people.

1. 0. The one idea
2. 1. The voting process (the part that took longest to get right)
3. 2. The registry (the file the queue reads and the bot reads back)
4. 3. Getting verdicts out of the phone (the integration gotcha)
5. 4. The notes section (their voice while they use the thing)
6. 5. Process rules that made the fleet work (hand these to the code bot verbatim)
7. 6. How to talk to the decider (the reply contract)
8. 7. The UI sauce (what the screens themselves must obey)
9. 8. Verification (how to know the screen works, not just that the code passes)
10. 9. Integration recipe for a personal-assistant UI (do this in order)
11. 10. Blind spots (things you do not know you do not know yet)
12. 11. If the assistant is a free chat bot (no code running, no files, no git)
13. Appendix A: the queue page, minimal working shape
14. Appendix B: the bot's read-back, in plain steps

0. The one idea

The human is not a user. The human is the **decider**. Every screen you build exists to do one of three things:

1. Show the decider a thing they have not judged yet.

2. Take one tap of judgement (up, down, a comment) and make the thing **leave**.
3. Get that judgement back to the bot as a file the bot can read.

Everything else is noise. If your assistant's UI does those three things well on a phone in sunlight, it is ahead of almost every "AI dashboard" in existence.

1. The voting process (the part that took longest to get right)

1.1 One queue, one place

There is exactly **one place** the decider votes. Not one page per feature, not a judge page per bot, not a modal here and a thumbs-up there. One queue, mixed kinds (a design, a paragraph, a sound, a name, a rule), newest first, each row with the **same three controls**: up, down, comment.

We learned this the hard way: eight separate judge pages, each built when it was "just one or two things", became a maze. The decider said: "one central tab where everything that is new goes, where I vote on it."

1.2 A vote consumes the item

The moment the decider votes, the row **leaves** and never comes back under that id. The decider's exact words: "after I vote on something it needs to stop presenting itself like I didn't just vote on it."

Two memories make this reliable:

- The device remembers the vote instantly (local storage) so the row vanishes on the tap.
- The registry file remembers it permanently once the export lands.

An item is consumed if it is in **either**. One memory was not enough: a rebuild of the page resurrected judged things.

1.3 A redo is a new item that quotes the kill

If the bot redoes something the decider killed, it comes back as a **new id** that points at the old one and says: "REDONE. You killed this on [date] because: [their exact words]." The decider is never judging blind, and never asked to re-judge the same id.

1.4 Notes are rulings

If the decider **said** something in a comment, that is the verdict. Build it. Never ask them to confirm their own words with a second thumb. Thumbs are for fresh, unseen candidates only.

1.5 Unjudged goes stale, and stale is dead

Bulk silence is a verdict. If the queue fills up and the decider does not touch a thing for a long stretch, that thing is dead. The bot does not nag. The queue must never become a debt.

1.6 Decide first, correct after (the biggest process lesson)

We turned the decider into an approvals queue and they hated it: "thumb thumb thumb, everything is a thumb." The fix flipped the default: **the bot decides, builds it, and puts it where the decider meets it; the decider corrects what they hate**. Only three things still go to the decider before building:

- identity and names they reserved for themselves,
- a genuine fork with no defensible default (pick one anyway, say why, build it, put it in the queue),
- anything they explicitly asked to see.

Every default the bot takes in the decider's place goes into the queue as its own row, one line with the *why*, so a rule can be knocked down the same way a design can.

1.7 Two kinds of item: aesthetic and philosophical

The queue holds **aesthetic** items (a look, a sound, a face, a line of text) and **philosophical** items (a rule, a default the bot took, a fork). Both get the same three controls. A down-vote on a philosophical item is a ruling; the bot rebuilds.

2. The registry (the file the queue reads and the bot reads back)

One JSON file, in a folder the web page can fetch. The page reads `items` and `verdicts`. Bots **append** to `items` and never edit another bot's object. Humans never edit it by hand.

```

{
  "version": 1,
  "items": [
    {
      "id": "ui-five-ways-the-first-screen-opens-9-21",
      "kind": "ui",
      "lane": "ui",
      "sha": "3992alea",
      "made": "9/21",
      "title": "FIVE WAYS THE FIRST SCREEN OPENS",
      "why": "The opening screen, five options at real phone size, beside the
flat one it replaces.",
      "show": { "how": "image", "src": "vote/first_screen_five_ways.png" }
    },
    {
      "id": "sounds-the-room-hum-redo-9-22",
      "kind": "redo",
      "lane": "sounds",
      "sha": "PENDING",
      "made": "9/22",
      "title": "THE ROOM HUM, REDONE",
      "why": "Quieter under the music; stays on when the song starts.",
      "show": { "how": "audio", "src": "vote/room_hum_v2.mp3" },
      "redoOf": "sounds-the-room-hum-9-21",
      "said": "too loud, sounds like a fridge"
    }
  ],
  "verdicts": [
    { "id": "sounds-the-room-hum-9-21", "vote": "down", "note": "too loud, sounds
like a fridge", "at": "2026-09-21T14:02:00Z" }
  ]
}

```

Field rules that saved us pain:

- `id` : a unique slug. **Never reused**. A redo is a new id.
- `kind` : a short fixed list your page knows how to show. Ours: `song sound face haircut outfit tile animation line ui verdict redo`. Add a kind by adding one word to the list and one renderer.
- `lane` : which bot made it. Lets the cook gate (section 5) count who is producing.
- `sha` : the commit the thing landed in, so anyone can go look. `PENDING` is allowed on first write and must be filled by a follow-up commit.
- `title` : plain words the decider reads on the row. Not a filename.
- `why` : one short line: what changed and what they are deciding.

- `show: {how, src}` where `how` is `page` | `image` | `clip` | `audio` | `text`. Paths are **relative to the page's folder**, never absolute. A broken image in the queue is a row that "shows nothing when tapped", the worst kind of bug.
- `redo0f + said: redos only`. `said` is the decider's own words, quoted back.

How the bot reads a verdict back: look in `verdicts` for your id. `up` = build it in, `down` = graveyard it with a one-paragraph post-mortem (what it was, why it died, what not to try again). Do not ask again.

The gate: an automated check plants a judged id in `verdicts` and proves the page refuses to render it. If that check does not exist, the rule does not exist.

3. Getting verdicts out of the phone (the integration gotcha)

A web page **cannot write a file**. So the verdicts have to leave the device by a road the decider can use with one thumb:

1. **Copy all.** One button puts every verdict (and every note) on the clipboard as plain text. The decider pastes it into the chat with the bot. This is the road they actually use.
2. **Export as** `.txt`. Never `.json`. The decider opens text files; they do not open JSON.
3. **Local storage, instantly.** The tap is remembered on the device before any export, so nothing is lost if they close the page.

The bot's job on receiving the paste: parse it, append to `verdicts` in the registry, commit, and write a human-readable record file the same turn. **The commit is the memory.** If it is not committed, it did not happen.

Plain-text export format that parses cleanly and reads like a receipt:

```
VERDICTS 9/21 14:02 (build 9/21c)
UP    ui-five-ways-the-first-screen-opens-9-21    "the third one, the others look
like apps"
DOWN  sounds-the-room-hum-9-21                    "too loud, sounds like a
fridge"
UP    people-the-names-at-your-door-9-21          ""
NOTE  [walk, block 12, 04:31, 88s in] the car still looks like dogshit at the far
zoom
```

Every line is one verdict or one note. Tabs or two-space padding. The bot regex is trivial.

4. The notes section (their voice while they use the thing)

The decider asked for "the tiniest button, top right, where as I'm playing I can write all my thoughts down and then resume." What we learned building it:

- **Tiny means footprint, never invisibility.** Our first version was a 5-pixel pencil. The decider could not find it, so it did not exist. It is now the word NOTES, small but readable, with a 44 px touch target.
- **Every note carries its own context** so the decider never types it: the build stamp, the screen or mode, where they were, the time into the session, and a small frame of what was on screen. "This looks bad" then says *what* looked bad. Before this, one comment ("the car looks bad") sent three bots guessing which zoom level was meant.
- **Opening the notes pauses the thing.** Inputs stop at the capture phase so nothing underneath reacts. A release event is never swallowed.
- **A tap outside the box must not throw away what they wrote.** Every other card can dismiss on a tap outside; the one place they write cannot.
- **Collapsible, and it stays there the whole session.** Tap open, tap closed.
- **Copy all** puts every note on the clipboard as text (same road as verdicts).

5. Process rules that made the fleet work (hand these to the code bot verbatim)

- **One word starts a round.** The decider types one word into any chat. The bot reads the shared rules page, finds its own section, and does its job. The decider never pastes context and never explains. If your assistant has more than one bot, give them one shared board.
- **Every sentence the decider says becomes a job before the reply ends.** Two-word bracketed labels ([copy notes] , [first sound]), each with what it is, why, and a ship test.

- **Claim before you work.** A bot marks the job CLAIMED and pushes that before starting, so two bots never build the same thing.
- **Newest date wins.** Rules accumulate; on any conflict the newest ruling wins, and the old one is marked dead the same turn. A contradiction between two live rules is a bug, not an interpretation.
- **A rule without a machine check is not enforced.** Every rule gets an automated gate. A gate is proven by **mutation**: break the thing on purpose and show the gate goes red. A gate that cannot go red is not a check.
- **Record the decider's exact words**, verbatim, in a file, the same turn. Rulings are derived from the words, and the words outlive the interpretation.
- **Re-read the shared file immediately before you write it.** We lost dozens of handoff notes because a bot read the file at the start of an hour-long round and wrote the whole file back at the end, deleting everything others had added. Read, then write, in the same breath. Better: one file per bot.
- **Ship every round.** The decider's standing complaint was small, timid turns. The other complaint, later, was bots that measured and checked all round and made nothing. The rule that fixed it: **a making bot ends every round with a real thing the decider can see or hear, in the queue.** Measuring rides beside the making, never instead of it. A gate counts it: a bot whose last cooked item is older than its last commit is red.
- **Reports lead with the thing** and where the decider can see it. Gates, numbers and measurements go at the bottom, in a proof line.

6. How to talk to the decider (the reply contract)

The decider reads on a phone, from the bottom of the screen up, at speed. Every reply:

1. **Lead with the answer or the outcome.** Never with process, never with a green gate.
2. **Plain words, short sentences, eighth-grade reading level.** No file paths, function names or gate names in the body. Numbers over adjectives.
3. **Name the place.** Every time you mention something they can look at, say which tab or screen it is in. If it is not reachable yet, say "NOT THERE YET" in those words. A thing they cannot reach does not exist to them.

4. **Never ask a technical or priority question.** If the bot could answer it by reading the code or by research, it is not a question, it is work not done. Legitimate questions: names the decider reserved, a genuine creative fork in plain words, anything they asked to see.
5. **One question maximum, bolded, with two or three lettered options (A/B/C)** and a default that wins if they say nothing. The realistic option leads.
6. **End every reply with two fixed blocks, last on the screen:**
 - **WHAT I NEED FROM YOU:** numbered, each answerable in one word, or "Nothing, I'm good."
 - A two-sentence bottom line: what you just did, and what they should do with it and why.
1. **No calendar talk.** To the decider the work is one continuous stream. Say "this round", "last round", "since your last message", never "yesterday" or "tomorrow".
2. **When they correct you: fix it, name the root cause in one line, move on.** No defence.

7. The UI sauce (what the screens themselves must obey)

7.1 Do not look vibe-coded

The decider's words: "People can look at it and be like, yep, that was coded." The tells, from research and then counted in our own files:

- Default fonts: Inter, Poppins, Space Grotesk, Geist, monospace for everything.
- Dark mode as the reflex.
- A one-pixel grey border on every card; rounded cards inside rounded cards.
- Purple-to-blue gradients, glow, "dark plus glow equals premium".
- Small uppercase letter-spaced labels on everything.
- Exactly three feature cards in a row.
- Emoji as icons inside buttons.

Why it happens: a model produces the statistical average of its training set instead of making a choice, so the result reads as nobody's decision.

The law: **none of the tells survive**. The look comes from the thing's world, not from a component library. An edge is a thing's edge. A label is painted, stamped, scratched or lit. A panel has thickness. Dark is a choice, not a default. **A gate counts the tells** in every shipped surface and fails on any of them.

7.2 Physical rules for a phone

- **Every control is at least 44 px** of touch target at 390 px width. "Tiny" is about the ink, never the target.
- **Sun mode by default**. Judged in daylight, on a phone. Night mode on a button.
- **A control under about 30 px tall is a readout, not a button**. Our automated tester was "tapping" 26 px labels and reporting dead buttons.
- **A dead button is indistinguishable from a close button**. A control check must prove the panel *stayed open* and *something changed*, not that a tap happened.
- **A card that promises and does nothing when tapped is the worst bug in the product**. Worse than a crash. The decider named it three times.
- **Nothing pops up**. No modal at the start, none on a timer, none "to explain". What the product has to say waits in a place the decider opens themselves.

7.3 Colour means ownership, never content

Two accents, and they carry **meaning**:

- **Gold** = you. The objective, the verb on the action button, anything you press, anything you are trying to do, the feedback on what you just did.
- **Cold (teal)** = the machine. Counts, timestamps, place names, anything the system hands you that you did not choose.
- A person is neither: plain ink.

Hard limit: no essential information by colour alone. Gold and cold say *whose* a thing is, never *what* it says. Every coloured value carries its own word beside it.

7.4 Lines, corners, contrast (numbers, not taste)

- The rule line is **2 px** and measured **3.78:1** against its panel. The hairline it replaced measured 1.22:1, which is nothing in sunlight.
- Corners: **cut** at 45 degrees (10 px deep on panels) or a 2 px object corner. Never a 5-8 px rounded card radius; that is the tell.
- **Pressed is a flip**, not a colour fade: the face inverts. It reads at arm's length.

7.5 Type

- **A quoted font family is a request for a file.** If no `@font-face` answers it, the browser silently serves something else. We shipped a month with no letters at all while every check was green. Gate it: every named family in the CSS must resolve to an embedded file.
- **Controls are drawn, not typed.** No font carries eight compass arrows in one weight; icons come back in two weights and one draws an envelope.
- **Test letters at the real size.** At 5 px our casing font's N and H differed by 10.6% of their ink, the closest pair in the alphabet. Render all 26 capitals at the shipped size and rank every pair; the eye on a zoomed screenshot lies.
- Three registers, never mixed: **casing** (stamped into the case), **screen** (lit glass, tabular), **body** (a person's sentence).

7.6 "3-D" as four concrete things (what we mean when we say it)

Not a drop shadow, not a rounded card. Every panel is built from exactly:

1. A **face** that catches one light, always above-left. Two lights read as a sticker.
2. A **lit rim** on the edge the light hits and a **dark base** on the edge it misses. Real value steps, never a grey hairline floating on nothing.
3. A **body**: the line under a panel is the side of the thing. Shadows are inset and outset on the same element.
4. A **short hard shadow** the object sits in, because the light is a fixture, not the sun.

Then it is delivered as pixels: every ramp is a **staircase of hard stops**, never a smooth gradient. All values come from one **skin file**, so a second theme swaps values and every panel is a different machine without one rule changing.

7.7 Interface findings we kept from two studies (adapt freely)

- The interface lives in a hue the world does not use.
- The readout says what it *does*, not what it *is*.
- The menu remembers where you were.
- The preview is the tutorial: show the result before the commit.
- Tell them the count; hide only the order.
- Everything is thick. The room is made of three materials and no more.

- Nothing waits: no spinner as a design element; a real loading screen with a **real** progress bar (bytes, steps, systems), never a timer. A bar that lies is the "promises and does nothing" bug in a new coat.

8. Verification (how to know the screen works, not just that the code passes)

- **Verify on the real surface.** Drive the real page on a phone-shaped profile (throttled CPU, 390x844). A unit test of the function is not the screen.
- **Print which file you opened.** Our driver opened a stale build by default and two bots measured the wrong file in one round, each getting a believable number with no error.
- **One run each side is not an A/B** on a browser-driven check with timeouts. A single red and a single green is a coin. Repeat, alternate, then decide.
- **An instrument that cannot say "I don't know" will say "no".** A bounded search that hits its limit looks exactly like a wall. Return three states: yes, no, unknown.
- **The first run is the cold run.** First load was 2x the rest. Warm up once, then measure; keep the cold number as a separate line.
- **A rare-event count cannot be pinned from one sample.** Accept a baseline from the worst of several runs, not from one lucky zero.
- **A red gate nobody reads is the same as no gate.** Walking was silent for six days while the footstep check said FAIL in plain words. Put the suite's red count in the shared board and make every bot say which reds are theirs.
- **A gate must never outrank a ruling.** When a check goes red, read what it asserts against the newest ruling before you read it against the code; it may be asserting a dead rule.
- **A number typed into a guard expires silently.** Derive limits from the thing they guard.
- **A ratchet, not a hope.** "Never worse than the last build" was a sentence for a week and the product got worse. Make it a machine: score the candidate build (time to tappable, seconds frozen, dead presses, page errors) and refuse the push if worse than the accepted baseline.

9. Integration recipe for a personal-assistant UI (do this in order)

1. **One registry JSON** in a folder the page can fetch. Schema in section 2. Commit it.
2. **One queue page.** Newest first. Each row: title, why, the `show` renderer for its kind, up, down, comment. Sun mode default. Comment box at the bottom, always. Every control 44 px.
3. **Consume on tap.** Local storage the instant they tap; the row leaves. The page filters out any id present in `verdicts` or local storage.
4. **Copy all + export .txt.** Section 3 format. No JSON export.
5. **Notes.** The word NOTES, top right, collapsible, pauses the thing, each note stamped with context, copy all.
6. **The bot's read-back.** On paste: parse, append to `verdicts`, commit, write a plain-text record, then act: `up` builds in, `down` graveyards with a post-mortem, a redo is a new id quoting `said`.
7. **The skin file.** Every colour, face and ramp in one tokens file. Customising the UI is editing values, never redrawing panels.
8. **The gates.** Vote-consumes (plant a judged id, prove it does not render). Tells counter (fonts, hairlines, radii, gradients, uppercase labels, emoji). Font-resolves (every named family has a file). Thumb (every control 44 px). Cook-every-round (per bot, last item vs last commit). Each mutation-proved.
9. **The reply contract** in the bot's system prompt, section 6, verbatim.
0. **Build stamp** on the first screen (date-letter plus headline) so the decider can see which build they are on. Pushing is not shipping; check the deploy actually finished.

10. Blind spots (things you do not know you do not know yet)

- **You will build the approvals queue and then resent it.** The fix is section 1.6: the bot decides and the decider corrects. Design the queue for *corrections*, not approvals.

- **The decider will say "in the demo" and mean "in the thing I use to judge".** We built the queue into the public build because of one transcribed word. Ask where they *judge*, put the queue there only.
- **Names and identity stay theirs.** Everything else is the bot's to decide. Put names in the queue as their own kind.
- **Voice-to-text garbles.** Never treat a garbled word as a new term. Decipher intent; record the raw words anyway.
- **A round marker does not exist in git.** If you want "per round" metrics, define what a round is (a commit range, a tag) or the gate will lie.
- **Shared files get eaten.** Registry, board, handoff: any file many bots append to will lose entries on a bad rebase. Resolve conflicts by taking the upstream copy and appending, then assert every id that was there is still there.
- **The decider reads bottom-up and never scrolls.** Anything above the fold of the last message does not exist. Same for the UI: the action is at the thumb, not the top.
- **"Cool" is a spec.** When they say "a cool loading screen, maybe even a bar if you're smart enough", the bar is required and it must read real progress. Treat adjectives as requirements with a measurable version.
- **The tiniest control still needs a 44 px target and a readable label.** "Tiny" is ink.
- **Sound needs a gesture.** A browser will not start audio before a tap. If your first screen blocks taps until loaded, the first sound can only play on the BEGIN tap. Design the BEGIN tap in from day one.
- **Green gates are never an argument and never lead a reply.** The decider does not care that tests pass. They care what they can see.

11. If the assistant is a free chat bot (no code running, no files, no git)

Everything above still works. The plumbing changes, because a free-tier chat bot cannot run code, cannot write files, cannot commit, and has a small memory window. The loop becomes **one chat, one page, copy and paste**. This is how to do it without losing any of the rules.

11.1 The page carries its own data

There is no server and no registry file to fetch. So the bot writes the queue as **one self-contained HTML page** (an artifact, or a file the decider saves) with the registry JSON **embedded inside it** as a `<script type="application/json" id="registry">` block. The page reads that block instead of fetching. Each round the bot regenerates the page with the new items added and the judged items removed. The page is the registry.

11.2 Verdicts go out by clipboard, come back by paste

The page's COPY ALL button is the only export. The decider pastes the text into the chat. The bot's read-back (Appendix B) runs as reasoning, not code: it lists what came in, marks each item judged, and emits the next page. Nothing else is needed. No JSON export ever: the decider will not open it, and the bot does not need it.

11.3 The board and the rules live in one pasted document

The bot forgets between sessions. So the decider keeps **one plain-text board** (this playbook's sections 1, 5, 6 boiled to a page, plus the open jobs with their two-word labels) and pastes it at the start of every session, or keeps it as the bot's project instructions if the plan offers that. Every sentence the decider says becomes a job on that board **in the bot's reply**, and the decider copies the updated board back. The board is the memory, the way a commit was for us.

11.4 One round is one message

Free tiers limit messages and context. So a round is **one message** from the decider that carries three things: the pasted board, the pasted verdicts and notes, and any new words. The bot answers with: the things it made (the page, the text), the updated board, and the two fixed blocks. Long back-and-forth burns the allowance; the round shape does not.

11.5 Keep the artifacts small

A page with images embedded as data URIs gets large and slow to regenerate. Prefer text items, CSS-drawn panels, and links to images the decider hosts elsewhere. Sounds: a free bot cannot make audio files; a "sound" item is a description plus, if the decider can host a file, a link. Say so on the row rather than pretending.

11.6 What the free tier cannot do, said plainly

- No automated gates. The mutation-proved checks in section 8 become a **checklist the bot runs by reading its own output** before it replies: is every control 44 px, does any named font lack a file, is any judged id still on the page, does every row's `show` point at something real. Weaker than a machine, better than nothing, and the checklist is short.
- No build stamp from a deploy. Put a **round number and date** at the top of the page by hand.
- No "cook every round" gate. The decider enforces it with one line at the end of each round: "what did you make this round that I can see?" If the answer is a list of checks, the round did not happen.
- No repo history. Keep the boards and the verdict pastes in a folder, one file per round, named by date. That folder is the record.

11.7 The three rules that matter most when there is no machine behind them

1. **A vote consumes the item** (the regenerated page must not carry a judged id).
2. **Decide, build, put it in the queue; correct after** (never an approvals queue).
3. **End every reply with WHAT I NEED FROM YOU and the two-sentence bottom line.**

If the bot keeps those three by habit, the rest can be added one rule at a time.

Appendix A: the queue page, minimal working shape

```

<!-- one queue, one registry, consume on tap, copy all -->
<main id="queue"></main>
<textarea id="comment" placeholder="say it plainly"></textarea>
<button id="copyAll">COPY ALL</button>
<script>
// free tier: the registry is embedded in this page, see section 11
// const reg = JSON.parse(document.getElementById('registry').textContent);
const REG = 'registry.json'; // hosted version: relative to this
page
const seen = new
Set(JSON.parse(localStorage.getItem('verdicts'))||'[]').map(v=>v.id));
fetch(REG).then(r=>r.json()).then(reg=>{
  const judged = new Set(reg.verdicts.map(v=>v.id));
  const rows = reg.items.filter(it=>!judged.has(it.id) &&
!seen.has(it.id)).reverse();
  for (const it of rows) render(it);
});
function vote(it, dir){
  const list = JSON.parse(localStorage.getItem('verdicts'))||'[]';
  list.push({id:it.id, vote:dir, note:document.querySelector('#comment').value,
at:new Date().toISOString()});
  localStorage.setItem('verdicts', JSON.stringify(list));
  document.getElementById(it.id).remove(); // a vote consumes the item
}
document.querySelector('#copyAll').onclick = ()=>{
  const list = JSON.parse(localStorage.getItem('verdicts'))||'[]';
  const txt = list.map(v=>`${v.vote.toUpperCase().padEnd(5)} ${v.id.padEnd(44)}
"${v.note||''}"`).join('\n');
  navigator.clipboard.writeText(`VERDICTS ${new Date().toLocaleString()}\n`+txt);
};
</script>

```

Every control drawn from the skin tokens, 44 px targets, sun mode default. The `render(it)` function switches on `it.show.how`: `image` draws it at real phone width, `audio` gets a play button (a sound you cannot hear in the queue is not cooked), `page` opens in place, `text` shows the words, `clip` plays on loop.

Appendix B: the bot's read-back, in plain steps

1. Paste arrives. Split lines. For each `UP|DOWN id "note"` append to `verdicts`.
2. For each `NOTE [...]` words write a record line with the context tag intact.
3. Commit the registry and a `.txt` record the same turn.
4. For every `up`: build it into the product; for every `down`: graveyard with a post-mortem.

5. Reply to the decider leading with what was built, then the two fixed blocks (section 6).

Every rule above has a story behind it and most have a number. Where a rule and your taste disagree, measure first.